

# 深圳大学实验报告

课程名称：移动互联网应用

实验项目名称：我的头条

学院：计算机与软件学院

专业：软件工程

指导教师：解为成

报告人：\_\_学号：\_\_班级：\_\_

实验时间：2026 年 4 月 27 日

实验报告提交时间：2026 年 5 月 1 日星

教务部制

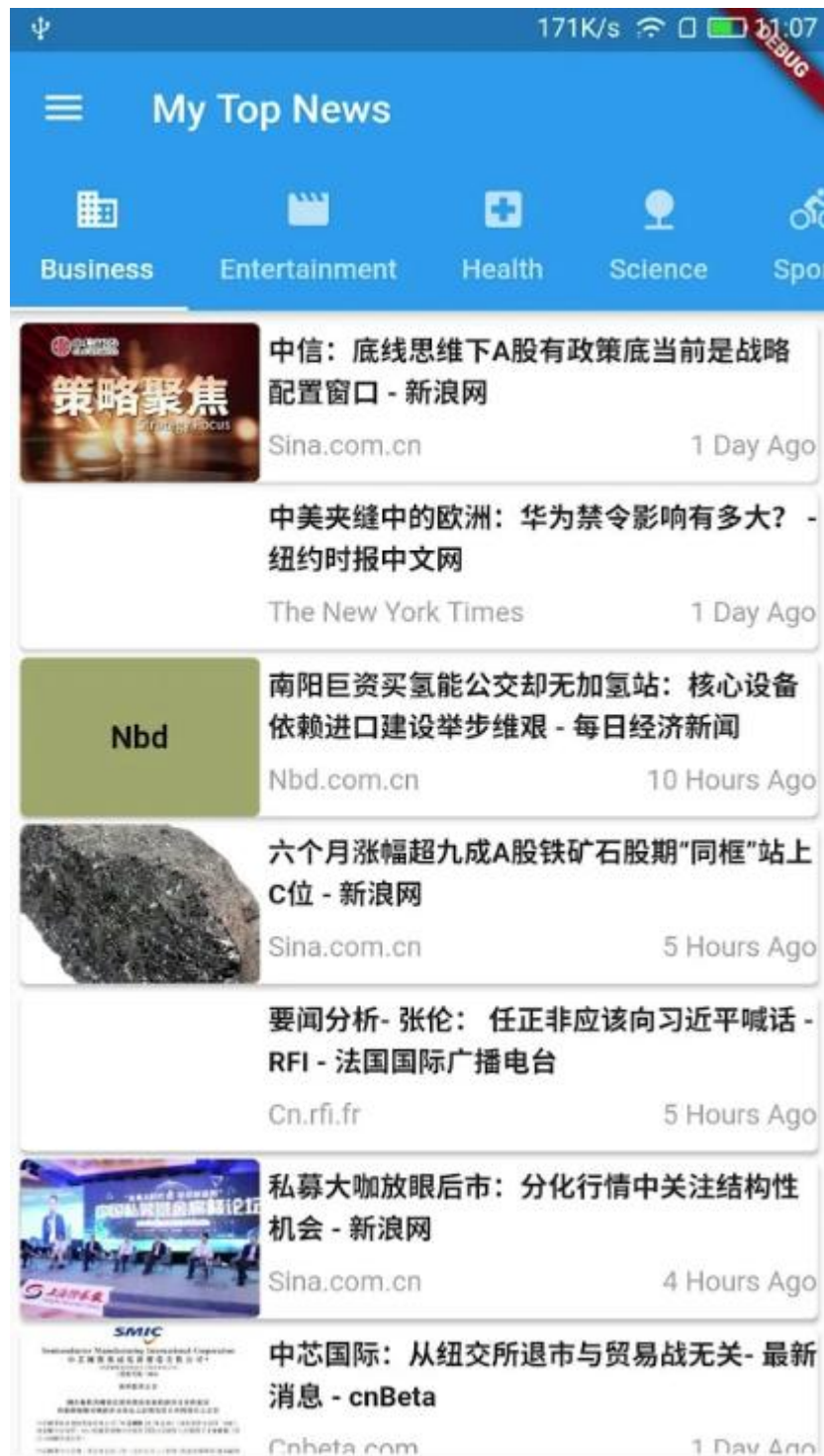
实验目的与要求:

**目的:** 掌握安卓中活动的编写、自定义用户界面的开发、能使用 HTTP 协议访问网络; 并能通过自学能适当完善该 APP 界面, 并使界面尽量美观。

**内容要求:**

1. 请尽量模拟如下 APP 界面的功能, 参考:

[https://play.google.com/store/apps/details?id=mark.h.my\\_news\\_app&hl=en\\_US](https://play.google.com/store/apps/details?id=mark.h.my_news_app&hl=en_US)



2. 该实现的界面应至少包含 3 个菜单, 分别展示个人 3 个方面的信息, 菜单之间要表现出一定的差异性; 每个菜单可以包含 2-5 个条目, 每个条目能响应个人某方面的偏好信

息；此外，如果是响应网页，需同时体现出 a) 采用浏览器浏览 与 b) 下载到本地 两种技术方案。

3. 尽量多的应用参考书《第一行代码 Android》第二版第 2 章（活动）、第 3 章（UI 开发）与第 9 章（网络技术）的访问网络相关知识；其中第 9 章技术点不作强行要求。

注意：

1. 实验报告中需要有功能的描述、实验结果的截屏图像及详细说明；
2. 也欢迎采用其它章节的知识点完成本次实验报告，如果实现的功能言之合理，会考虑酌情加分；
3. 尽量删除空白页。

方法、步骤：

#### 1. UI 架构设计

采用 Material Design 组件库中的 TabLayout 配合 ViewPager2 实现多菜单切换。

采用 ConstraintLayout 实现复杂的图文混排列表项，提高界面的适配性和渲染效率。

采用 RecyclerView 替代传统的 ListView，实现新闻列表的高效滚动和内存复用。

#### 2. 数据传递及页面跳转

采用显式 Intent 实现应用内 MainActivity 到 DetailActivity 的跳转，并利用 putExtra() 方法传递标题和 URL 数据。

采用隐式 Intent (Intent.ACTION\_VIEW) 调用系统底层组件（如外部浏览器）。

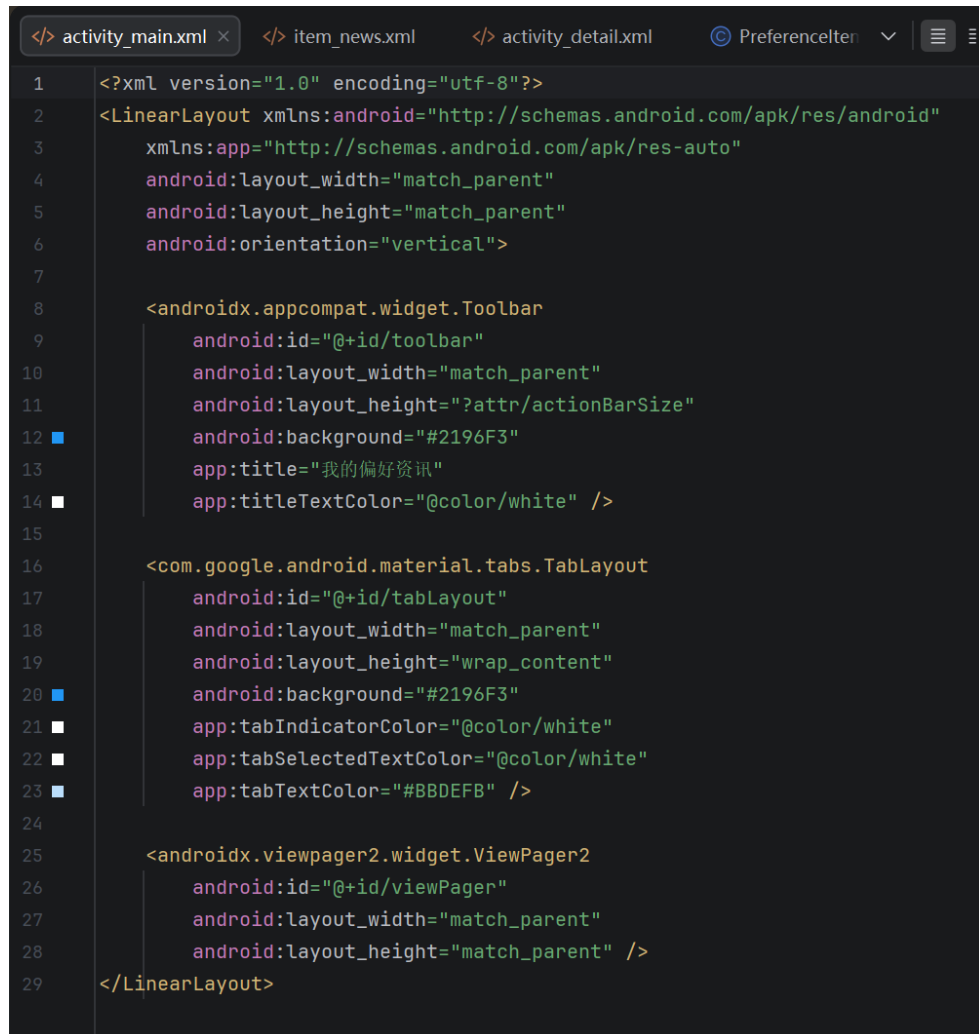
#### 3. 网络请求

采用 Java 标准的 HttpURLConnection 在子线程（Thread）中发起 HTTP GET 请求，获取网页源码后，通过 runOnUiThread() 方法切换回主线程更新 UI 界面。

实验过程及内容:

## 1.UI 界面开发

activity\_main.xml: 垂直线性布局, 自上而下放置了自定义蓝底白字的 Toolbar(标题栏)、TabLayout(分类菜单标签) 以及 ViewPager2(滑动内容容器)。



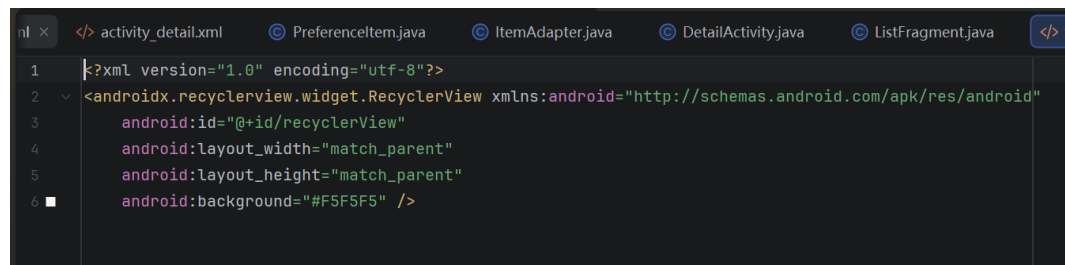
```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <androidx.appcompat.widget.Toolbar
        android:id="@+id/toolbar"
        android:layout_width="match_parent"
        android:layout_height="?attr/actionBarSize"
        android:background="#2196F3"
        app:title="我的偏好资讯"
        app:titleTextColor="@color/white" />

    <com.google.android.material.tabs.TabLayout
        android:id="@+id/tabLayout"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="#2196F3"
        app:tabIndicatorColor="@color/white"
        app:tabSelectedTextColor="@color/white"
        app:tabTextColor="#BBDEFB" />

    <androidx.viewpager2.widget.ViewPager2
        android:id="@+id/viewPager"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />
</LinearLayout>
```

fragment\_list.xml: 作为 ViewPager 的子页面容器, 内部仅包含一个全屏的 RecyclerView 控件。



```
<?xml version="1.0" encoding="utf-8"?>
<androidx.recyclerview.widget.RecyclerView xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/recyclerView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#F5F5F5" />
```

item\_news.xml: 列表单项的自定义布局。使用 ConstraintLayout 为根节点，左侧放置一个色块 ImageView 模拟封面，右侧利用约束关系错落有致地排布了标题（Title）、来源（Source）和时间（Time）三个 TextView。

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:app="http://schemas.android.com/apk/res-auto"
4      android:layout_width="match_parent"
5      android:layout_height="wrap_content"
6      android:padding="8dp"
7      android:layout_margin="4dp"
8      android:background="@android:color/white"
9      android:elevation="2dp"> <!-- 简单模拟卡片阴影 -->
10
11      <ImageView
12          android:id="@+id/img_cover"
13          android:layout_width="100dp"
14          android:layout_height="80dp"
15          android:scaleType="centerCrop"
16          android:background="#A1857E"
17          app:layout_constraintStart_toStartOf="parent"
18          app:layout_constraintTop_toTopOf="parent" />
19
20      <TextView
21          android:id="@+id/tv_title"
22          android:layout_width="0dp"
23          android:layout_height="wrap_content"
24          android:layout_marginStart="8dp"
25          android:textColor="#000000"
26          android:textSize="16sp"
27          android:textStyle="bold"
28          app:layout_constraintEnd_toEndOf="parent"
29          app:layout_constraintStart_toEndOf="@+id/img_cover"
30          app:layout_constraintTop_toTopOf="parent" />
31
32      <TextView
33          android:id="@+id/tv_source"
34          android:layout_width="wrap_content"
35          android:layout_height="wrap_content"
36          android:textColor="#888888"
37          android:textSize="12sp"
38          app:layout_constraintBottom_toBottomOf="@+id/img_cover"
39          app:layout_constraintStart_toStartOf="@+id/tv_title" />
40
41      <TextView
42          android:id="@+id/tv_time"
43          android:layout_width="wrap_content"
44          android:layout_height="wrap_content"
45          android:textColor="#888888"
46          android:textSize="12sp"
47          app:layout_constraintBottom_toBottomOf="@+id/img_cover"
48          app:layout_constraintEnd_toEndOf="parent" />
49
50  </androidx.constraintlayout.widget.ConstraintLayout>
```

activity\_detail.xml: 详情页布局。顶部展示大号标题，中间水平等宽放置两个 Button（分别对应浏览器浏览和下载源码功能），下方使用 ScrollView 包裹一个 TextView，用于展示长篇幅的 HTML 源码。

```
nl  </> activity_detail.xml ×  PreferenceItem.java  ItemAdapter.java  DetailActivity.java
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="match_parent"
4      android:layout_height="match_parent"
5      android:orientation="vertical"
6      android:padding="16dp">
7
8      <TextView
9          android:id="@+id/detail_title"
10         android:layout_width="match_parent"
11         android:layout_height="wrap_content"
12         android:textSize="20sp"
13         android:textStyle="bold"
14         android:layout_marginBottom="16dp"/>
15
16     <LinearLayout
17         android:layout_width="match_parent"
18         android:layout_height="wrap_content"
19         android:orientation="horizontal">
20
21         <Button
22             android:id="@+id/btn_browser"
23             android:layout_width="0dp"
24             android:layout_height="wrap_content"
25             android:layout_weight="1"
26             android:text="a) 浏览器浏览" />
27
28         <Button
29             android:id="@+id/btn_download"
30             android:layout_width="0dp"
31             android:layout_height="wrap_content"
32             android:layout_weight="1"
33             android:text="b) 下载源码到本地" />
34     </LinearLayout>
35
36     <ScrollView
37         android:layout_width="match_parent"
38         android:layout_height="match_parent"
39         android:layout_marginTop="16dp"
40         android:background="#EEEEEE">
41
42         <TextView
43             android:id="@+id/tv_content"
44             android:layout_width="match_parent"
45             android:layout_height="wrap_content"
46             android:text="点击上方按钮操作..."
47             android:padding="8dp"/>
48     </ScrollView>
49 </LinearLayout>
```

## 2. 数据绑定 (PreferenceItem.java)

构建实体类 PreferenceItem.java: 封装了单条资讯所需的数据结构, 包含 title (标题)、source (来源)、time (时间) 和 url (目标网址), 并提供 Getter 方法。

编写适配器 ItemAdapter.java: 继承自 RecyclerView.Adapter。

在 onCreateViewHolder 中动态加载 item\_news.xml 布局。

核心逻辑: 在此处为每个列表项的 itemView 绑定点击事件 (setOnClickListener)。当用户点击某一条目时, 获取当前位置的数据, 创建 Intent 携带 title 和 url 数据, 跳转至 DetailActivity。

在 onBindViewHolder 中将实体类的数据实际渲染到 UI 控件上。

```
1 package com.example.mynews;
2
3 // usages
4 public class PreferenceItem {
5     // 2 usages
6     private String title;
7     // 2 usages
8     private String source;
9     // 2 usages
10    private String time;
11    // 2 usages
12    private String url; // 用于网络请求跳转
13
14    // 6 usages
15    public PreferenceItem(String title, String source, String time, String url) {
16        this.title = title;
17        this.source = source;
18        this.time = time;
19        this.url = url;
20    }
21
22    // Getter 方法
23    // 2 usages
24    public String getTitle() { return title; }
25    // 1 usage
26    public String getSource() { return source; }
27    // 1 usage
28    public String getTime() { return time; }
29    // 1 usage
30    public String getUrl() { return url; }
31 }
```

## 3. 分类信息的差异化展示 (ListFragment.java)

实现了 Fragment 的 onCreateView 生命周期方法, 初始化 RecyclerView 并设置 LinearLayoutManager。通过构造函数接收 category 参数。在代码中编写逻辑分支, 根据传入的是“科技控”、“旅行记”还是“读书角”, 向列表中注入不同主题、不同来源的模拟数据 (PreferenceItem), 从而表现出各菜单之间个人偏好的差异性。

```
1 public class ListFragment extends Fragment {
2     // 4 usages
3     private String category;
4
5     // 1 usage
6     public ListFragment(String category) { this.category = category; }
7
8     // 2 usages
9     @Nullable
10    @Override
11    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container, @Nullable Bundle savedInstanceState) {
12        View view = inflater.inflate(R.layout.fragment_list, container, attachToRoot: false); // 需要创建一个简单的包含RecyclerView的布局
13        RecyclerView recyclerView = view.findViewById(R.id.recyclerView);
14
15        LinearLayoutManager layoutManager = new LinearLayoutManager(getContext());
16        recyclerView.setLayoutManager(layoutManager);
17
18        // 模拟生成不同方面的个人偏好数据
19        List<PreferenceItem> itemList = new ArrayList<>();
20
21        if ("科技控".equals(category)) {
22            itemList.add(new PreferenceItem("Android 14 新特性详解", "source: CSDN", "time: 2小时", "url: https://developer.android.google.cn/about/versions/android-14"));
23            itemList.add(new PreferenceItem("AI 大模型发展趋势分析", "source: 科技精英", "time: 5小时", "url: https://www.bing.com/"));
24        } else if ("旅行记".equals(category)) {
25            itemList.add(new PreferenceItem("西藏自驾游全攻略", "source: 携程游记", "time: 1天", "url: https://www.ctrip.com/"));
26            itemList.add(new PreferenceItem("寻找隐秘的江南水乡", "source: 马蜂窝", "time: 3天", "url: https://www.mafengwo.cn/"));
27        } else if ("读书角".equals(category)) {
28            itemList.add(new PreferenceItem("《第一行代码》读书笔记", "source: 个人博客", "time: 10分钟", "url: https://github.com/"));
29            itemList.add(new PreferenceItem("年度十佳科幻小说推荐", "source: 豆瓣阅读", "time: 1周", "url: https://book.douban.com/"));
30        }
31
32        ItemAdapter adapter = new ItemAdapter(itemList);
33        recyclerView.setAdapter(adapter);
34        return view;
35    }
36 }
```





```

15 public class DetailActivity extends AppCompatActivity {
16     3 usages
17     private TextView tvContent;
18     3 usages
19     private String targetUrl;
20
21     @Override
22     protected void onCreate(Bundle savedInstanceState) {
23         super.onCreate(savedInstanceState);
24         setContentView(R.layout.activity_detail);
25
26         TextView tvTitle = findViewById(R.id.detail_title);
27         tvContent = findViewById(R.id.tv_content);
28         Button btnBrowser = findViewById(R.id.btn_browser);
29         Button btnDownload = findViewById(R.id.btn_download);
30
31         // 接收上一级传来的数据
32         String title = getIntent().getStringExtra( name: "title");
33         targetUrl = getIntent().getStringExtra( name: "url");
34         tvTitle.setText(title);
35
36         // 方案A: 隐式Intent调用系统浏览器浏览
37         btnBrowser.setOnClickListener( View v -> {
38             Intent intent = new Intent(Intent.ACTION_VIEW);
39             intent.setData(Uri.parse(targetUrl));
40             startActivity(intent);
41         });
42
43         // 方案B: HttpURLConnection 下载网页源码到本地TextView展示
44         btnDownload.setOnClickListener( View v -> sendRequestWithHttpURLConnection());
45     }

```

```

45     1 usage
46     private void sendRequestWithHttpURLConnection() {
47         tvContent.setText("正在加载...");
48         new Thread(() -> {
49             HttpURLConnection connection = null;
50             BufferedReader reader = null;
51             try {
52                 URL url = new URL(targetUrl);
53                 connection = (HttpURLConnection) url.openConnection();
54                 connection.setRequestMethod("GET");
55                 connection.setConnectTimeout(8000);
56                 connection.setReadTimeout(8000);
57                 InputStream in = connection.getInputStream();
58                 reader = new BufferedReader(new InputStreamReader(in));
59                 StringBuilder response = new StringBuilder();
60                 String line;
61                 while ((line = reader.readLine()) != null) {
62                     response.append(line).append("\n");
63                 }
64                 showResponse(response.toString());
65             } catch (Exception e) {
66                 e.printStackTrace();
67                 showResponse("加载失败: " + e.getMessage());
68             } finally {
69                 if (reader != null) {
70                     try { reader.close(); } catch (Exception e) { e.printStackTrace(); }
71                 }
72                 if (connection != null) { connection.disconnect(); }
73             }
74         }).start();
75     }
76
77     // UI更新在主线程
78     2 usages
79     private void showResponse(final String response) {
80         runOnUiThread(() -> tvContent.setText(response));
81     }

```

## 6.数据桥梁与交互核心：(ItemAdapter.java)

构建 ViewHolder 缓存机制：在 ItemAdapter 内部定义了静态内部类 ViewHolder，继承自 RecyclerView.ViewHolder。在其构造函数中，通过 findViewById 一次性获取并缓存了单项布局 (item\_news.xml) 中的标题、来源、时间等控件引用。这极大地避免了列表滑动时频繁实例化控件带来的性能开销。

重写 onCreateViewHolder() 方法 (视图创建与事件绑定)：

首先利用 LayoutInflater 将 R.layout.item\_news 布局文件实例化为具体的 View 对象，并将其包装成 ViewHolder 返回。

交互逻辑核心实现：在此方法内为 holder.itemView 设置了 setOnClickListener 监听器。当用户点击某一条新闻时，通过 holder.getBindingAdapterPosition() 获取当前准确的点击索引，从 mList 中精准提取对应的 PreferenceItem 对象；随后构建 Intent，利用 putExtra() 将该条目的 title (用于显示) 和 url (用于网络请求) 传递给 DetailActivity 并启动跳转。

重写 onBindViewHolder() 方法 (数据渲染)：该方法负责将真实数据绑定到 UI 上。系统在列表滚动时会不断回调此方法，程序根据传入的 position 取出特定的实体对象，利用 Getter 方法获取文本，并调用 setText() 实时更新 ViewHolder 中缓存的各个 UI 控件的内容。

重写 getItemCount() 方法：直接返回内部数据集的大小 (mItemList.size())，告知底层的 RecyclerView 需要为其分配多少个基础列表项空间。

```
12 public class ItemAdapter extends RecyclerView.Adapter<ItemAdapter.ViewHolder> {
13     4 usages
14     private List<PreferenceItem> mList;
15
16     5 usages
17     static class ViewHolder extends RecyclerView.ViewHolder {
18         2 usages
19         TextView tvTitle, tvSource, tvTime;
20         2 usages
21         View itemView;
22
23         1 usage
24         public ViewHolder(View view) {
25             super(view);
26             itemView = view;
27             tvTitle = view.findViewById(R.id.tv_title);
28             tvSource = view.findViewById(R.id.tv_source);
29             tvTime = view.findViewById(R.id.tv_time);
30         }
31     }
32
33     1 usage
34     public ItemAdapter(List<PreferenceItem> itemList) { mList = itemList; }
35
36     @NonNull
37     @Override
38     public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
39         View view = LayoutInflater.from(parent.getContext()).inflate(R.layout.item_news, parent, attachToRoot: false);
40         final ViewHolder holder = new ViewHolder(view);
41
42         // 活动跳转。点击列表项跳转到详情页
43         holder.itemView.setOnClickListener( View v -> {
44             int position = holder.getAdapterPosition();
45             PreferenceItem item = mList.get(position);
46             Intent intent = new Intent(parent.getContext(), DetailActivity.class);
47             intent.putExtra( name: "title", item.getTitle());
48             intent.putExtra( name: "url", item.getUrl());
49             parent.getContext().startActivity(intent);
50         });
51         return holder;
52     }
53
54     @Override
55     public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
56         PreferenceItem item = mList.get(position);
57         holder.tvTitle.setText(item.getTitle());
58         holder.tvSource.setText(item.getSource());
59         holder.tvTime.setText(item.getTime());
60     }
61
62     @Override
63     public int getItemCount() { return mList.size(); }
64 }
```

数据处理分析：

1. 运行程序，可以看到主页页面布局



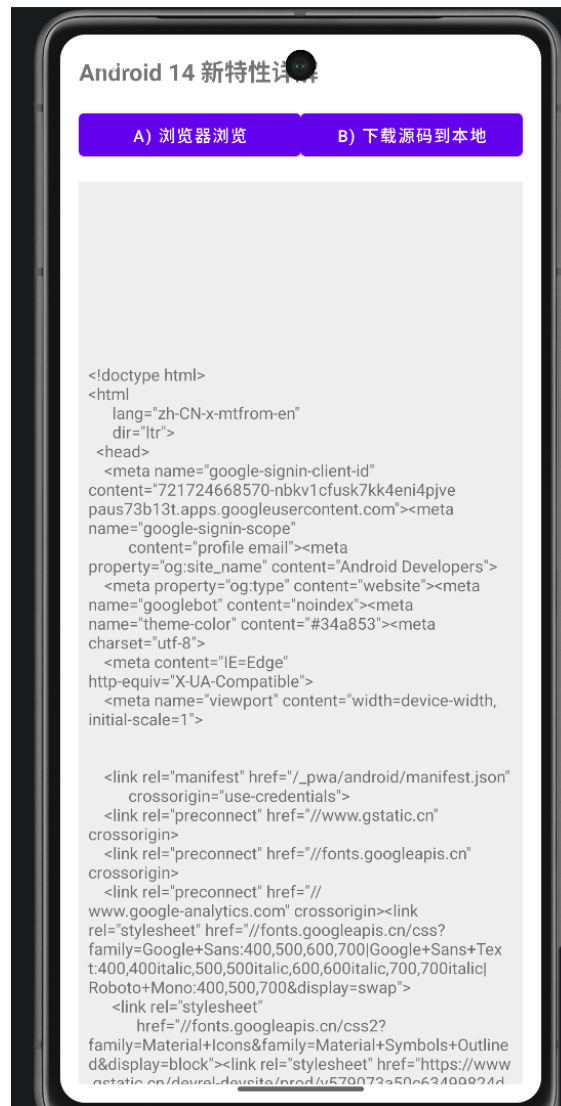
2. 点击板块按钮，可以切换对应板块



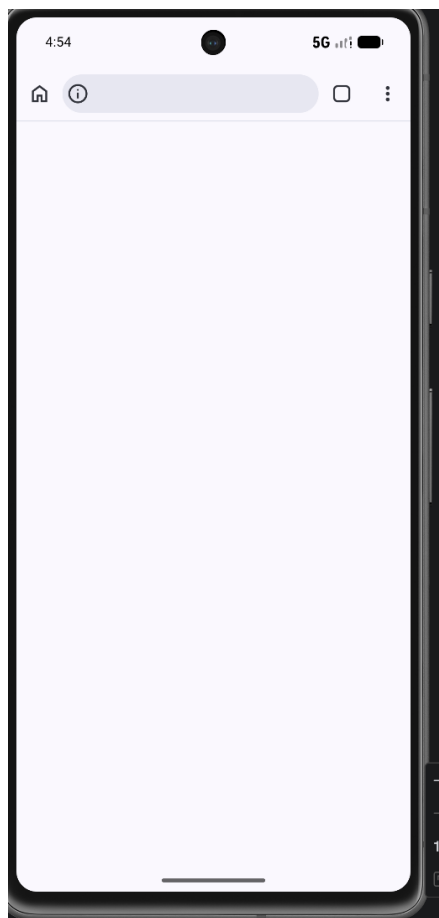
3. 点击相关头条内容，可以进入到页面查看



4. 在相关头条内点击“下载源码到本地”，加载后在下方可以看到内容源码



5. 点击“浏览器浏览”，能够成功跳转至浏览器



但由于模拟器未配置浏览器，所以只能跳转呼出浏览器，并进入不到页面。

深圳大学学生实验报告用纸

实验结论：

成功实现了多分类资讯展示、页面跳转、网络访问与本地下载等核心功能。通过本次实验，我掌握了 Android 基础 UI 开发、多页面交互与网络编程的核心方法，理解了 Material Design 组件的使用规范，提升了代码编写与问题调试能力，为后续 Android 复杂应用开发奠定了坚实基础。

指导教师批阅意见：

成绩评定：

指导教师签字：解为成  
2026 年 5 月 日

备注：

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。  
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。